



Heaven's Light is Our Guide

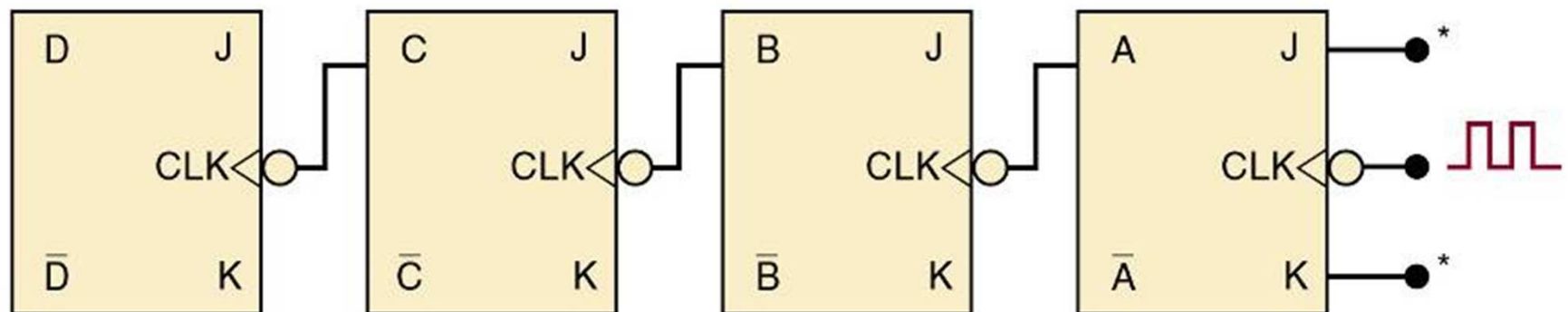
Department of Computer Science & Engineering
Rajshahi University of Engineering & Technology, Bangladesh

Course Code: CSE 2203
Course Title: Digital Techniques

Presented by,
Prof. Dr. Boshir Ahmed

Asynchronous (Ripple) Counters

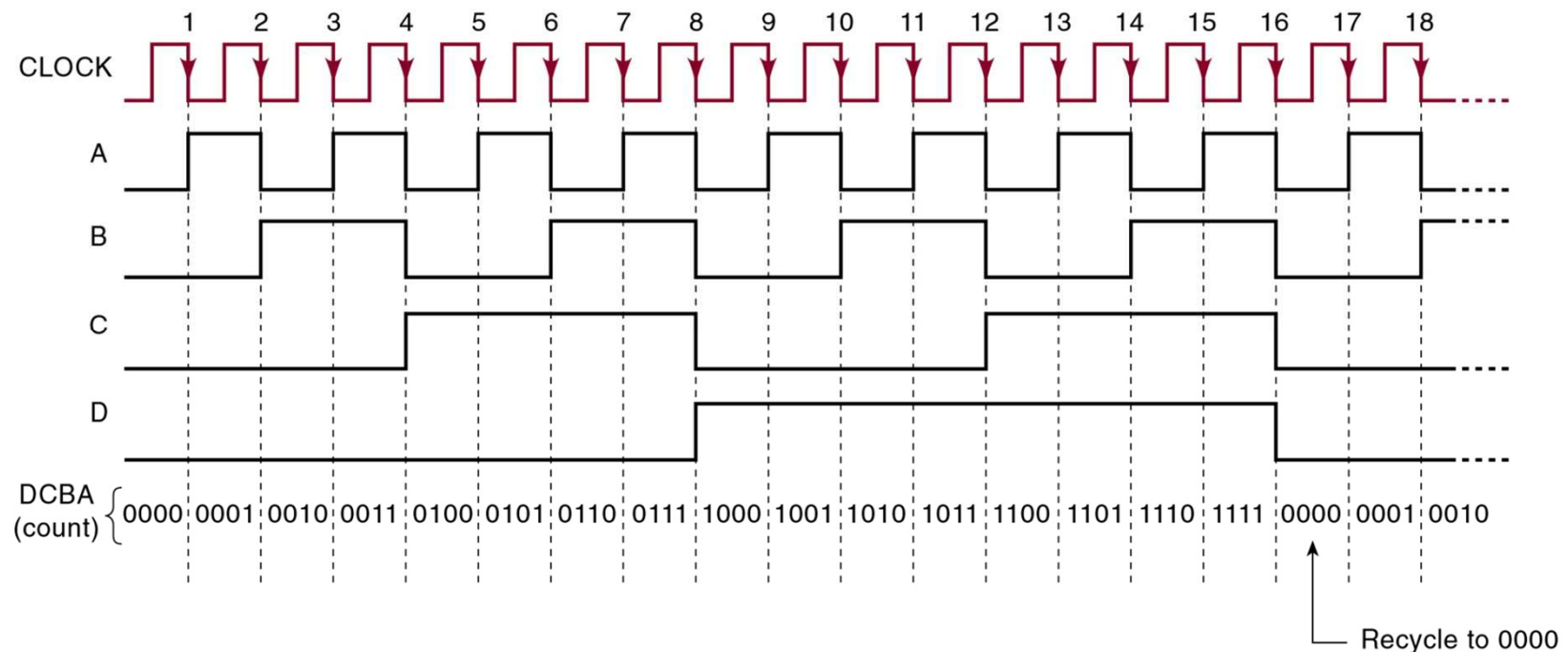
- Review of four bit counter operation:
 - Clock is applied only *CLK* input to FF *A*.
 - J & K are HIGH in all FFs.
 - Output of FF *A* is *CLK* of FF *B*, etc.
 - FF outputs *D*, *C*, *B*, and *A*, are a 4-bit binary number with *D* as the MSB.



*All J and K inputs
assumed to be 1.

Asynchronous (Ripple) Counters

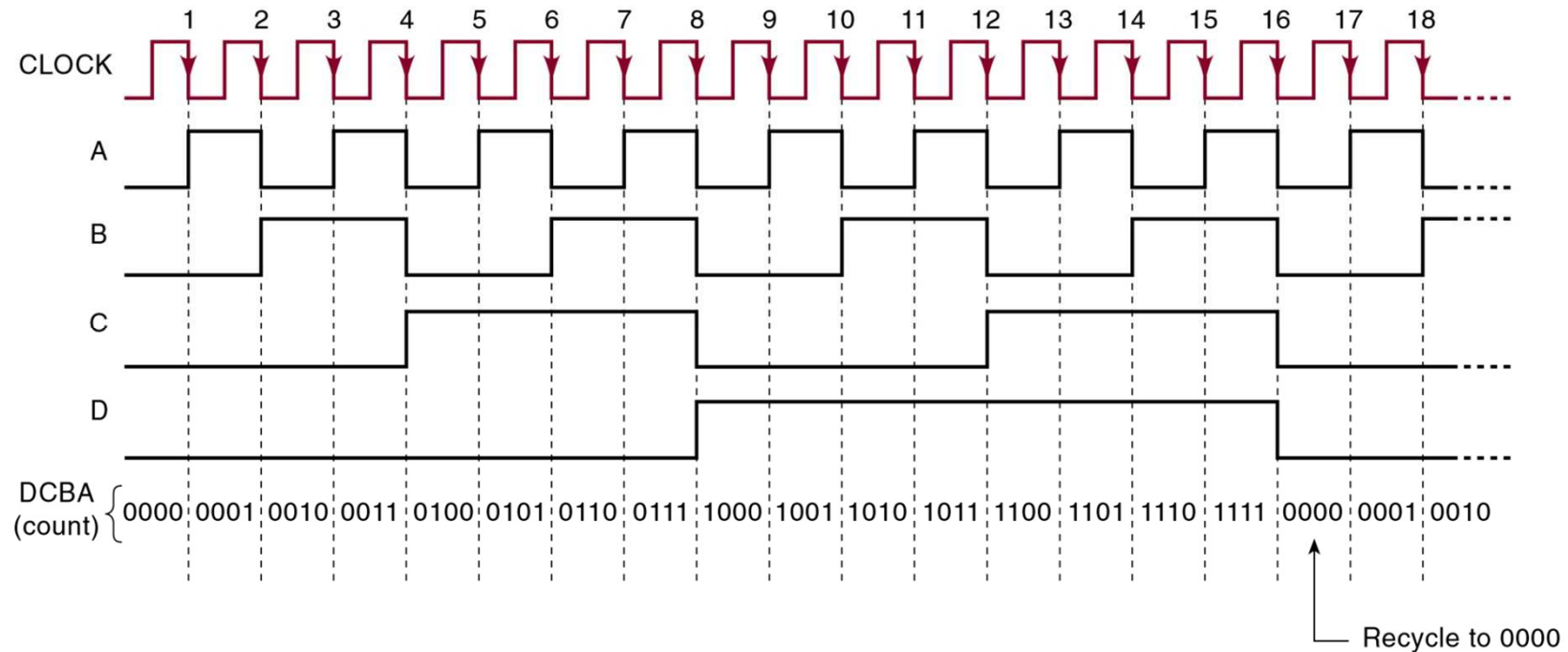
- Review of four bit counter operation:
 - After the NGT of the 15th clock pulse, the counter FFs are *recycled* back to 0000.



An asynchronous counter—state is *not* changed in exact synchronism with the clock.

Asynchronous (Ripple) Counters – MOD Number

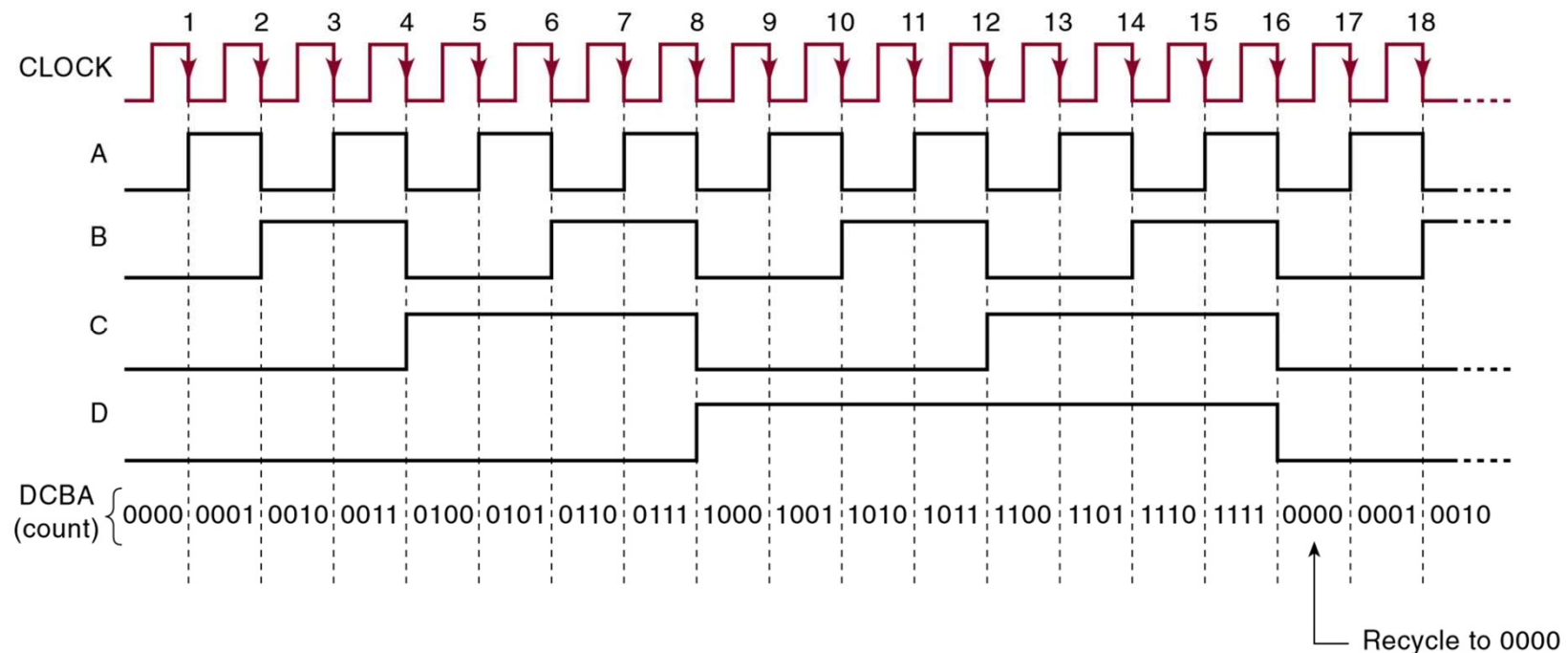
- MOD number is equal to the number of states the counter goes through before recycling.



Adding FFs will increase the MOD number.

Asynchronous (Ripple) Counters – MOD Number

- Frequency division – each FF will have an output frequency of 1/2 the input.



Output frequency of the last counter FF will be clock frequency divided by MOD of the counter.

Asynchronous (Ripple) Counters – Signal Flow

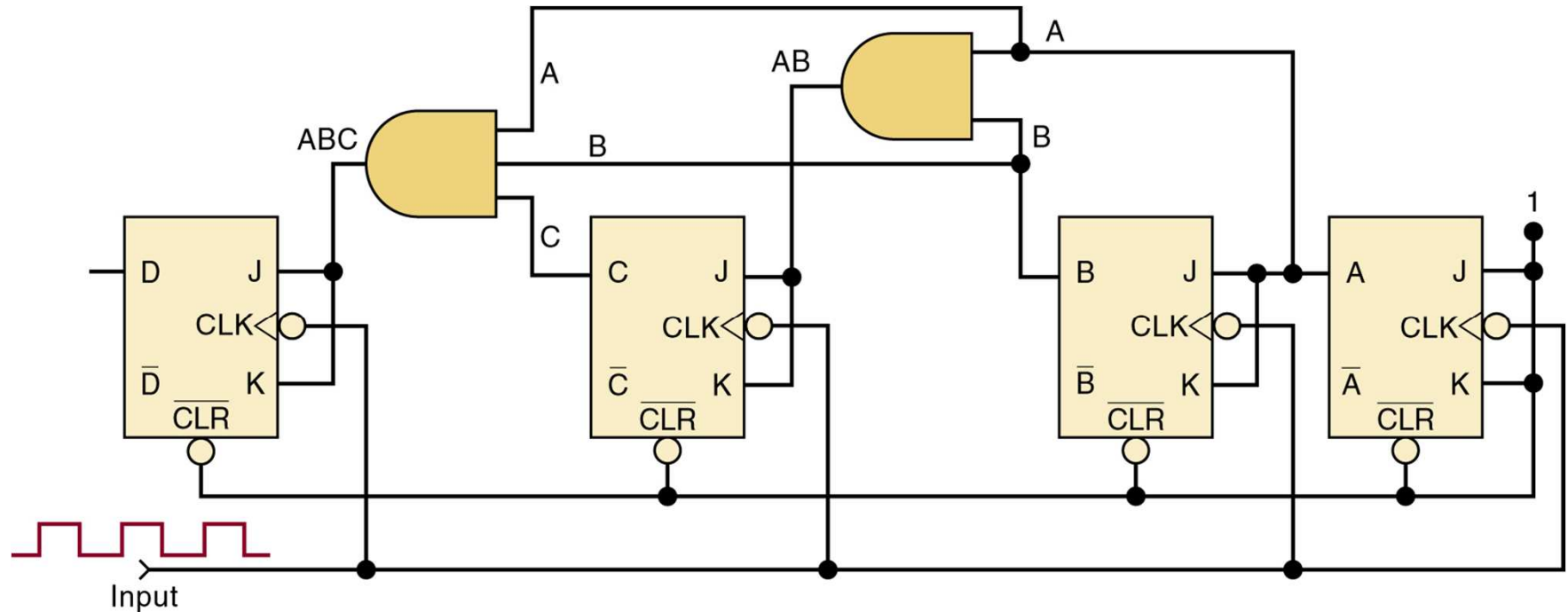
- Schematics are normally drawn from left to right.
 - Counters will be drawn from right to left so that the MSB and LSB appear in the appropriate positions.

Propagation Delay in Ripple Counters

- Ripple counters are simple—requiring the fewest components to produce a given operation.
 - Cumulative propagation delay can cause problems at high frequencies.
- If the period between input pulses is made longer than the total propagation delay of the counter, problems can be avoided
 - For proper operation: $T_{\text{clock}} \geq N \times t_{\text{pd}}$
 - Maximum frequency: $F_{\text{max}} = 1/(N \times t_{\text{pd}})$

Synchronous (Parallel) Counters

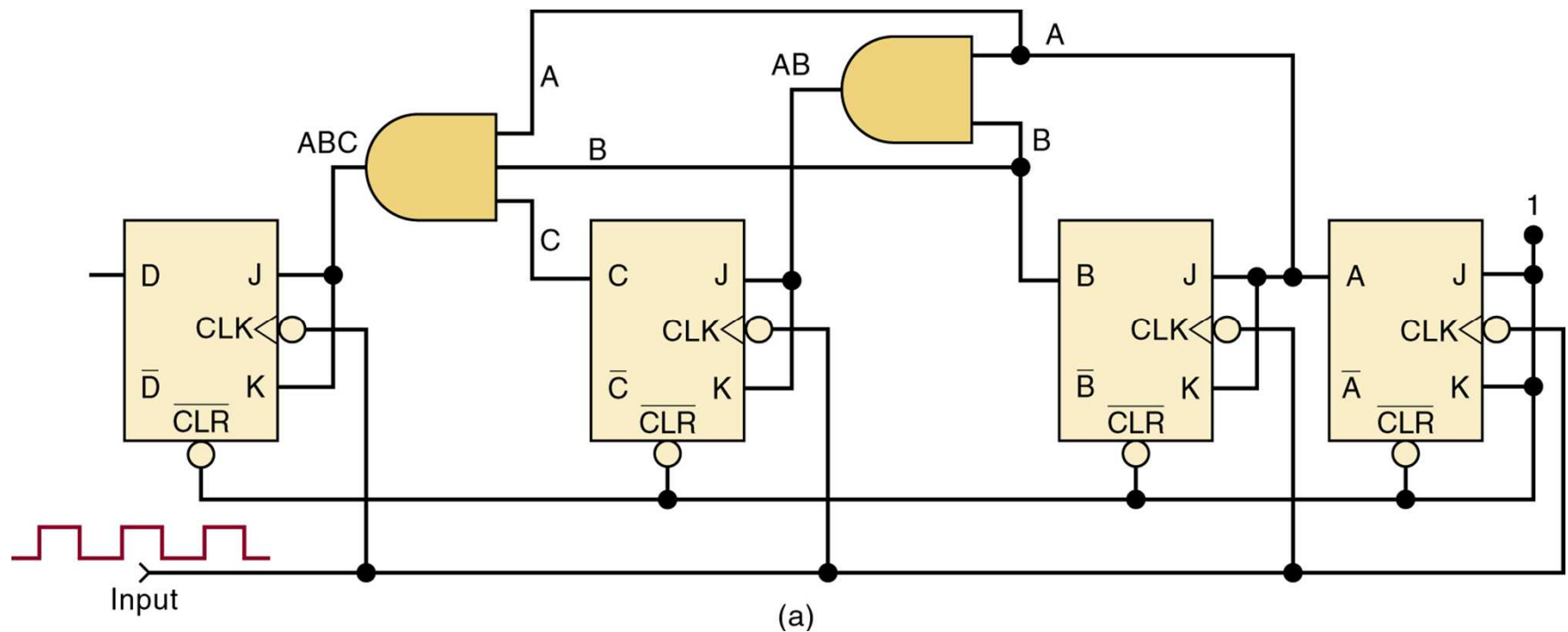
- In **synchronous** or **parallel counters**, all FFs are triggered simultaneously (in parallel) by the clock.



Synchronous counters can operate at much higher frequencies than asynchronous counters.

Synchronous (Parallel) Counters

- Each FF has J & K inputs which are HIGH only when outputs of **all lower-order FFs are HIGH**.



The total propagation delay will be the *same* for *any* number of FFs.

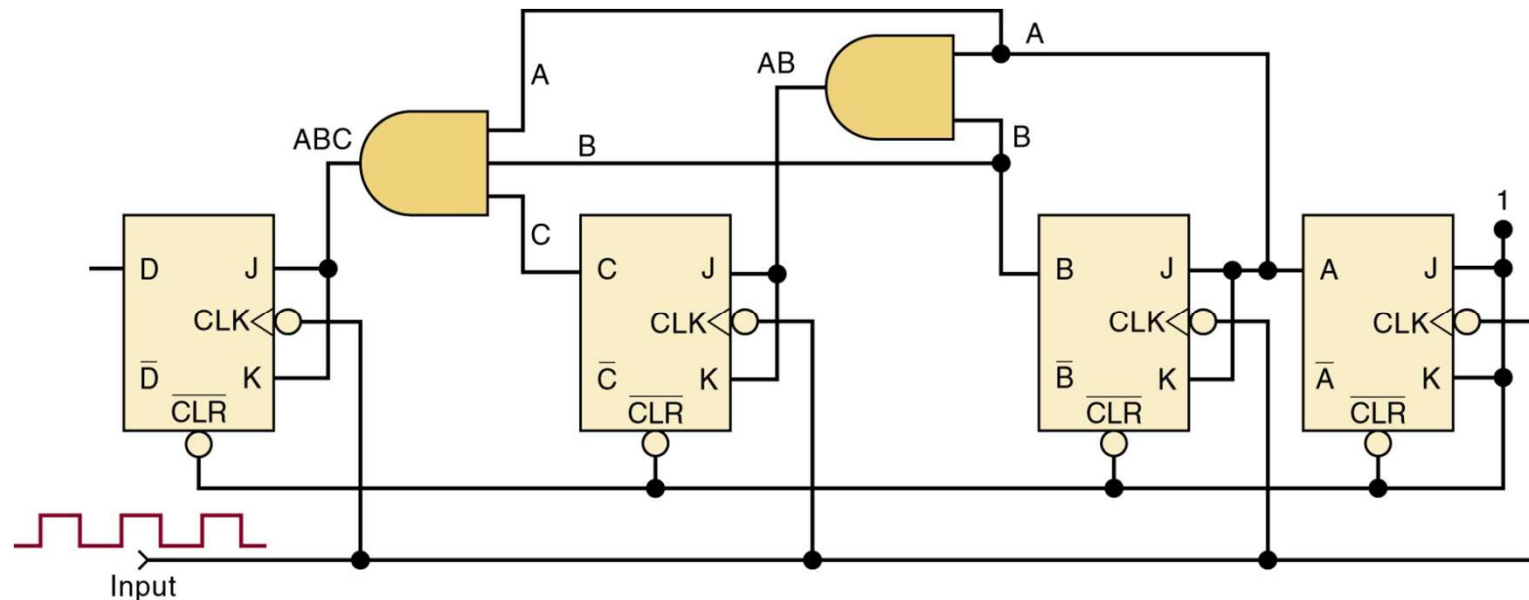
Synchronous (Parallel) Counters

For this circuit to count properly, on a given NGT of the clock, only those FF that are supposed to toggle on that NGT should have $J = K = 1$ when NGT occurs.

Count		D	C	B	A
0		0	0	0	0
1		0	0	0	1
2		0	0	1	0
3		0	0	1	1
4		0	1	0	0
5		0	1	0	1
6		0	1	1	0
7		0	1	1	1
8		1	0	0	0
9		1	0	0	1
10		1	0	1	0
11		1	0	1	1
12		1	1	0	0
13		1	1	0	1
14		1	1	1	0
15		1	1	1	1
0		0	0	0	0
.		.	.	.	.
.		.	.	.	.
.		.	etc.	.	.

Counters with MOD Number $< 2^N$

- The basic synchronous counter shown is limited to MOD numbers that are equal to 2^N .
 - Where N is the number of FFs.



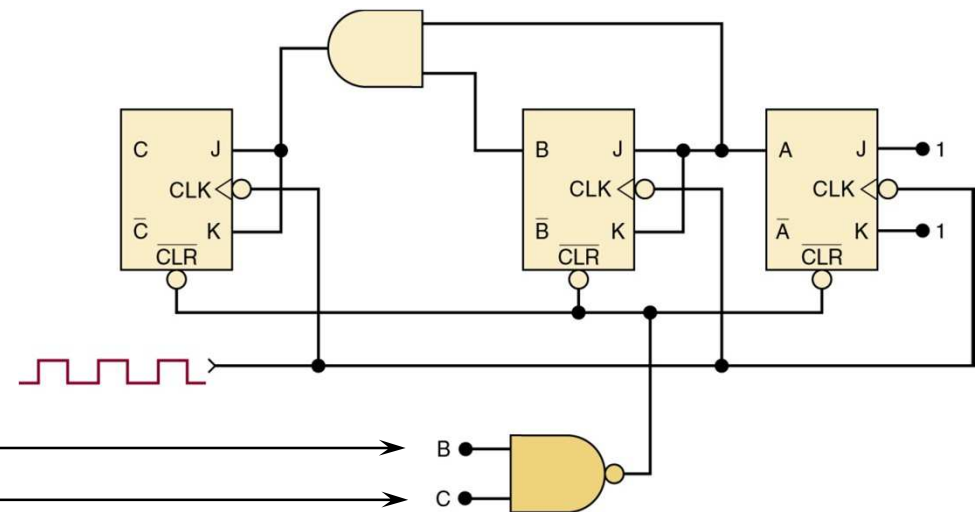
- MOD-6 counter produced by clearing a MOD-8 counter when a count of six (110) occurs.



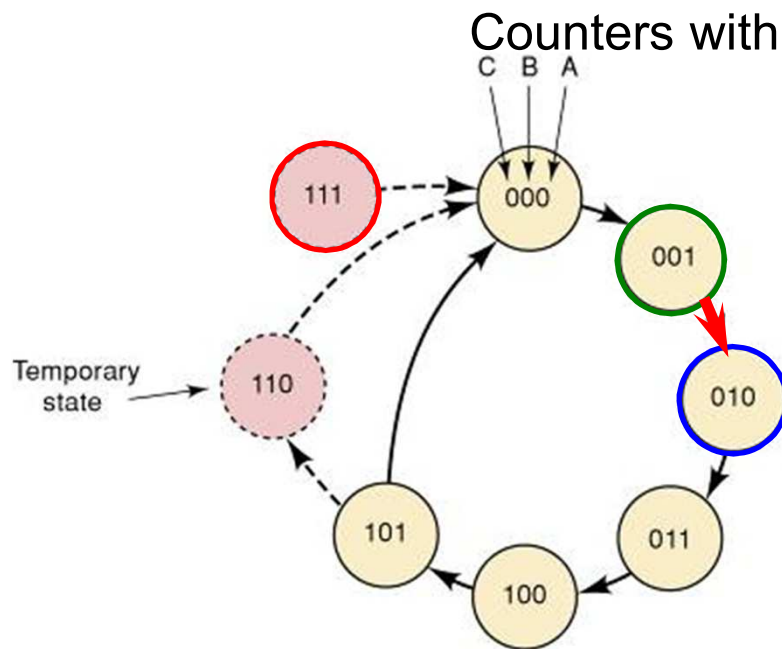
Counters with MOD Number $< 2^N$

- Changing the MOD number.
 - Find the smallest MOD required so that 2^N is less than or equal to the requirement.
 - Connect a **NAND** gate to the asynchronous CLEAR inputs of all FFs.

Determine which FFs are HIGH at the desired count and connect the outputs of these FFs to the **NAND** gate inputs.



State transition diagram for the MOD-6 counter

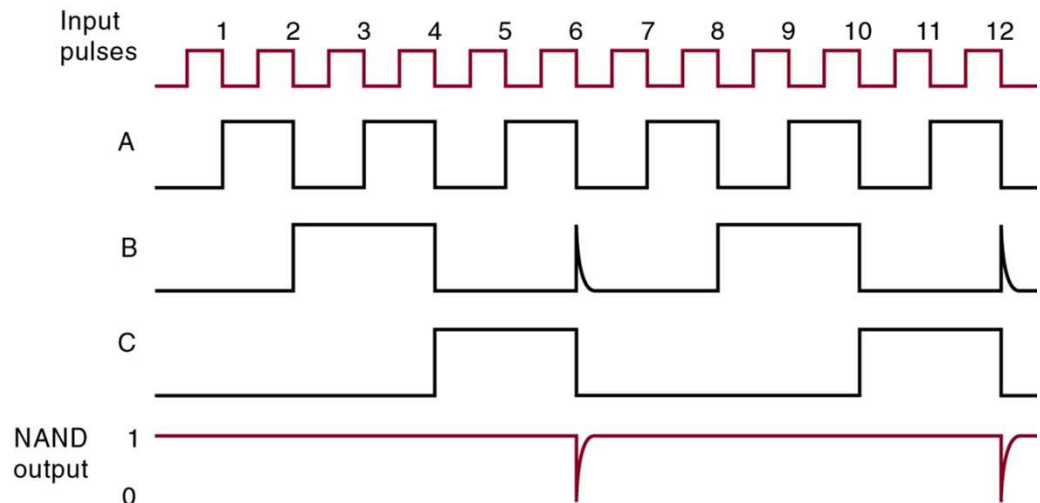


There is no arrow to the 111 state as the counter can *never advance* to that state.

The 111 state *can* occur on power-up when the FFs come up in random states.

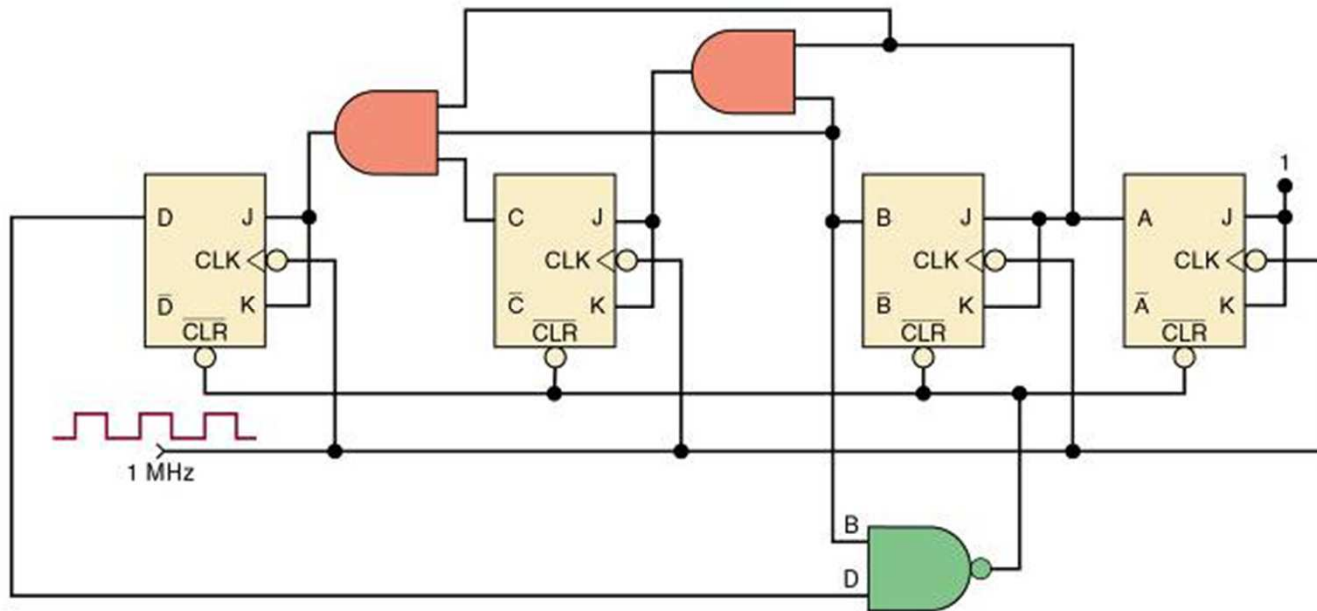
Each circle represents one of the possible counter states.

Arrows indicate how one state changes to another—in response to an input clock pulse.



Counters with MOD Number $< 2^N$

- Decade counters are widely used for counting events, and displaying results in decimal form.
 - A decade counter is any counter with 10 distinct states, regardless of the sequence.
 - A BCD counter is a decade counter that counts from binary 0000 to 1001.

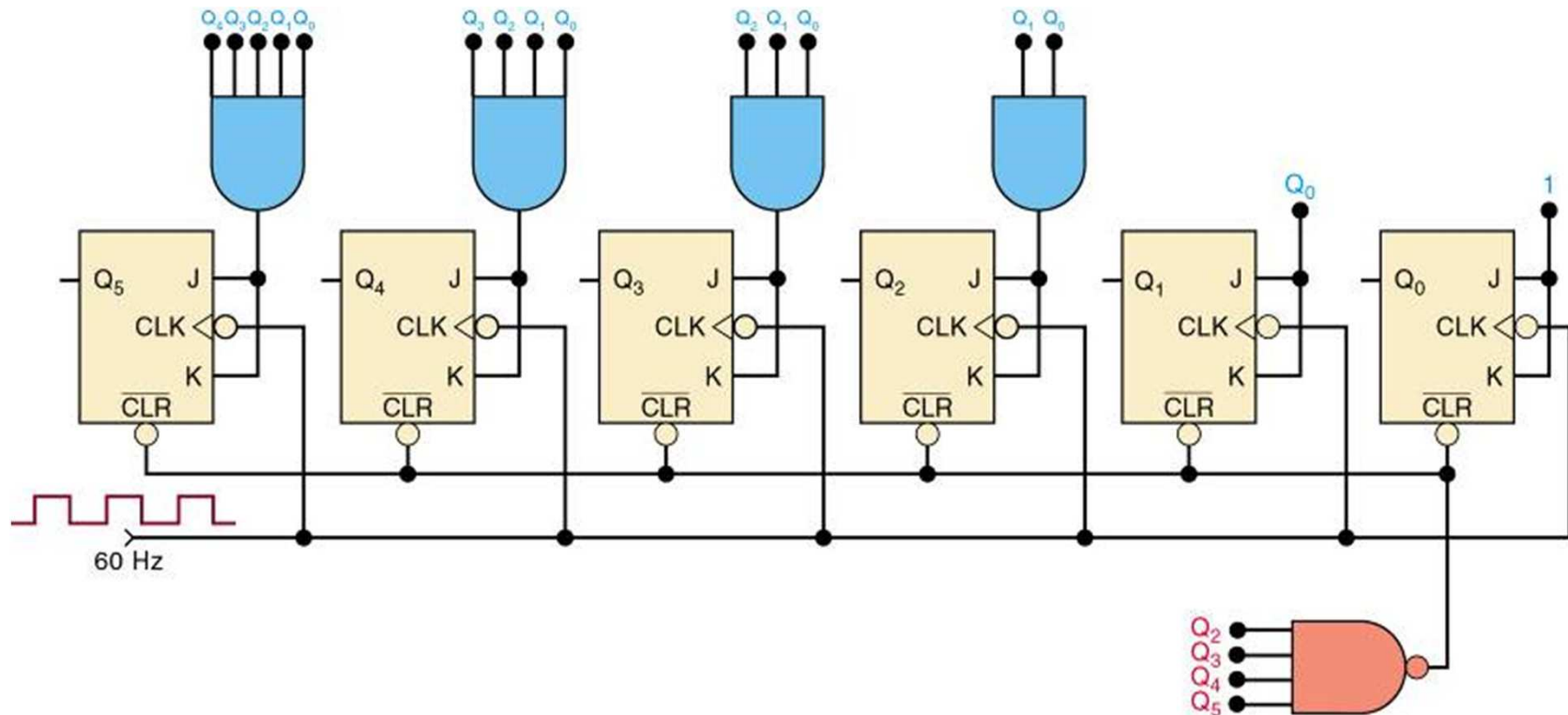


Any MOD-10 counter is a decade counter.

Counters with MOD Number $< 2^N$

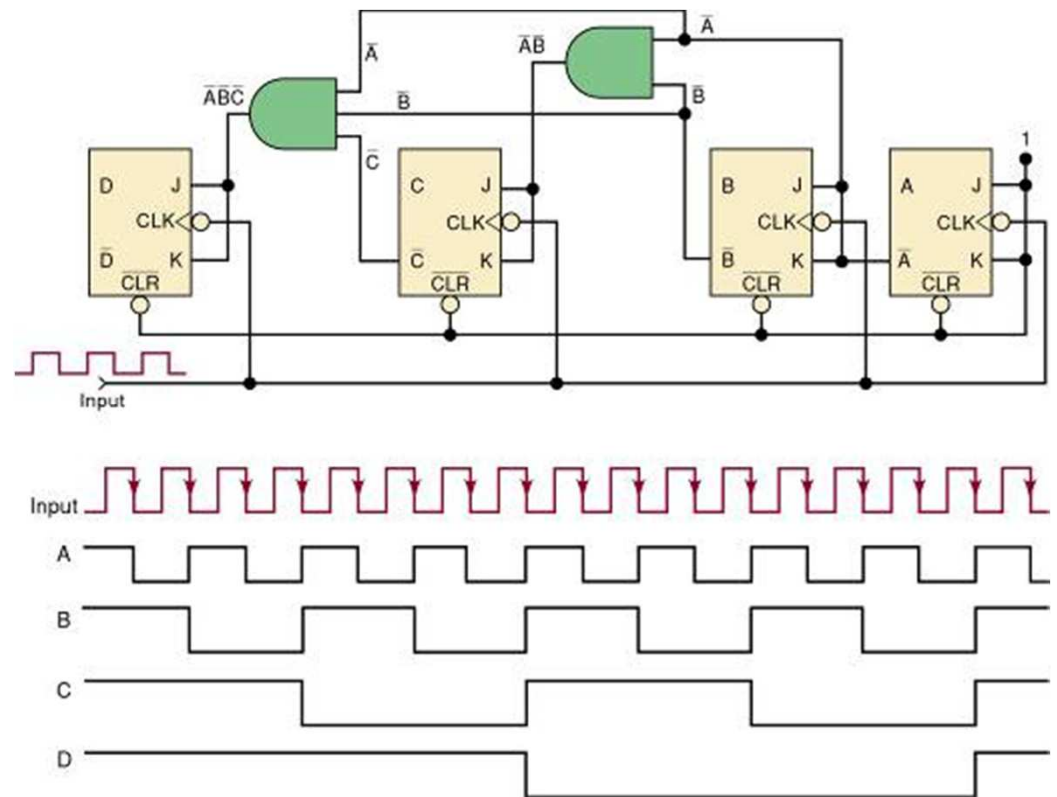
Mod-60 Counter

$$(60)_{10} = (111100)_2 = (Q_5 Q_4 Q_3 Q_2 Q_1 Q_0)$$



Synchronous Down and Up/Down Counters

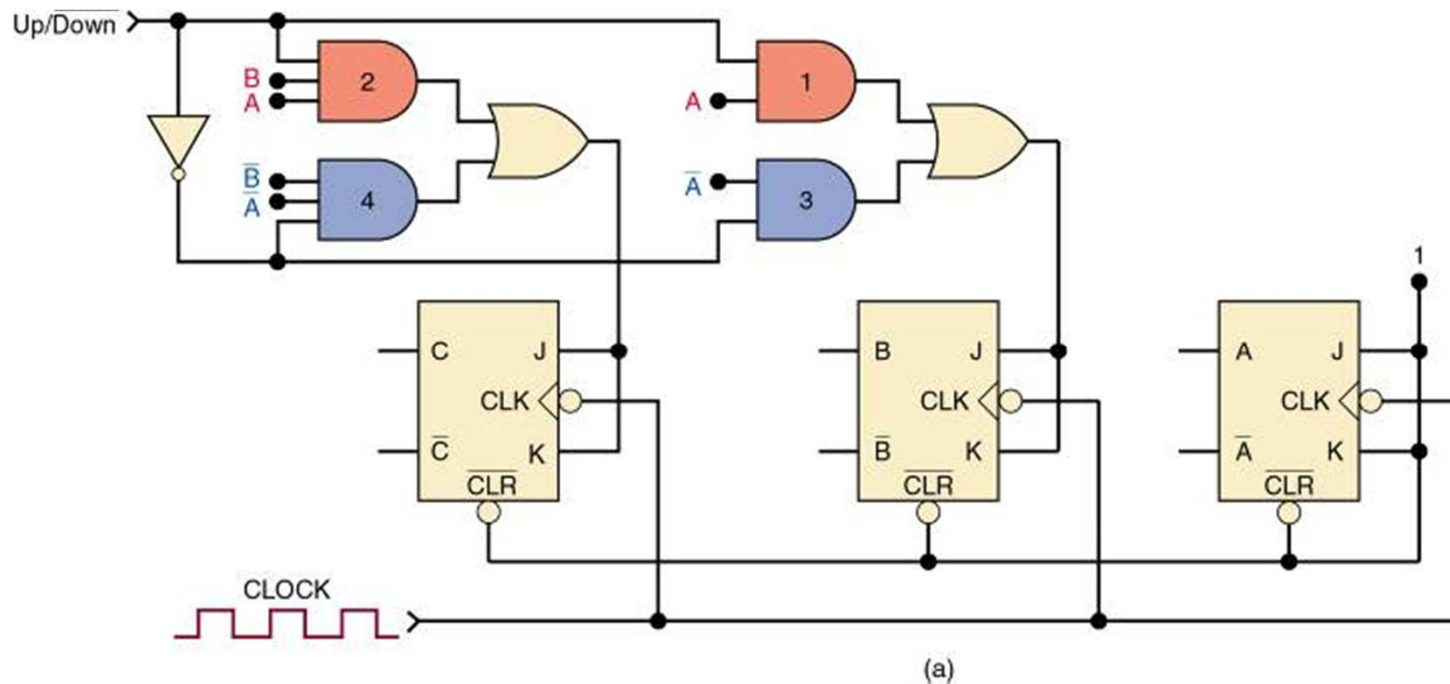
- A synchronous **down counter** is constructed in a similar manner to an up counter.
 - It uses the inverted FF outputs to control the higher-order J, K inputs.



**Synchronous, MOD-16,
down counter and output
waveforms.**

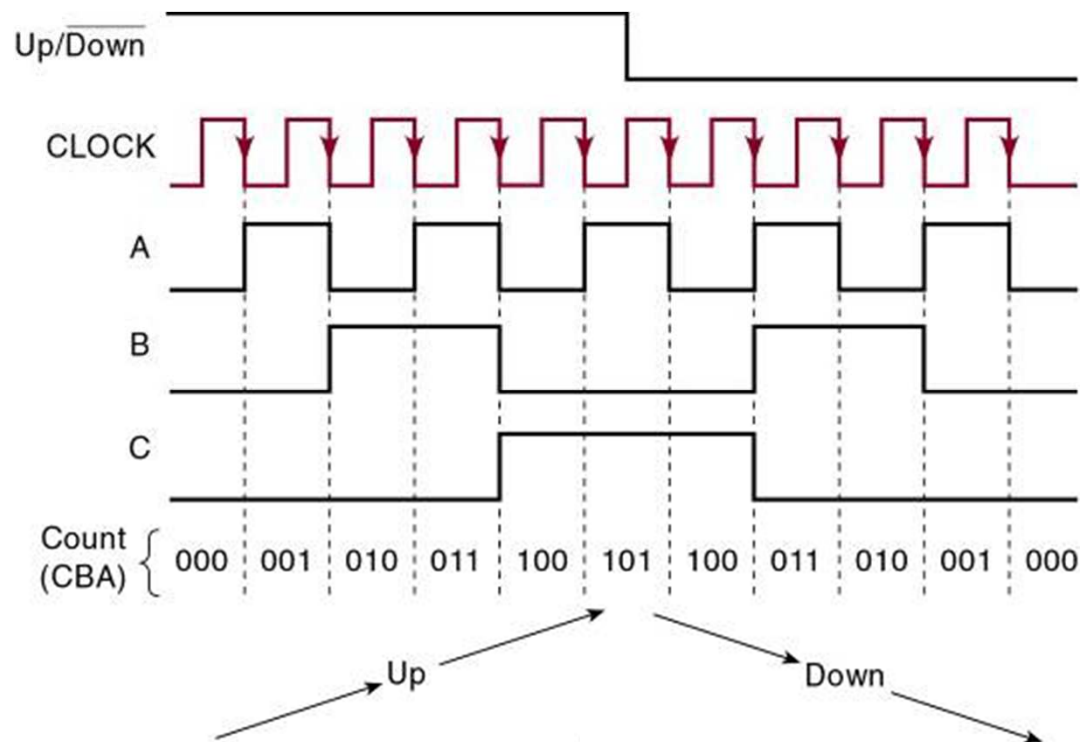
Synchronous Down and Up/Down Counters

- In a parallel **up/down counter**, the control input controls the values fed to the J and K inputs of the successive FFs.
 - The *normal* FF outputs or the *inverted* FF outputs.



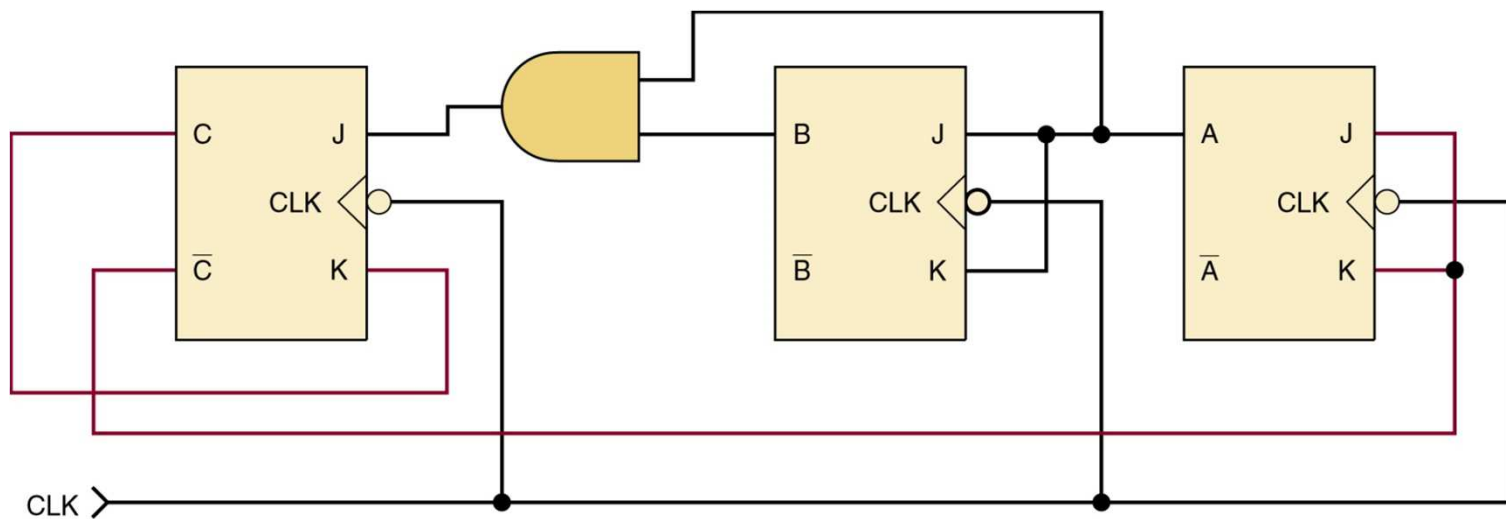
Synchronous Down and Up/Down Counters

- For the first five clock pulses, $\text{Up}/\overline{\text{Down}} = 1$.
 - The counter counts up.
- For the last five pulses, $\text{Up}/\overline{\text{Down}} = 0$.
 - The counter counts down.



Analyzing Synchronous Counters

Synchronous up counter.



Control input expressions.

$$J_C = A \cdot B$$

$$K_C = C$$

$$J_B = K_B = A$$

$$J_A = K_A = \overline{C}$$

- Synchronous counters can be custom-designed to generate any desired count sequence.



Control input expressions.

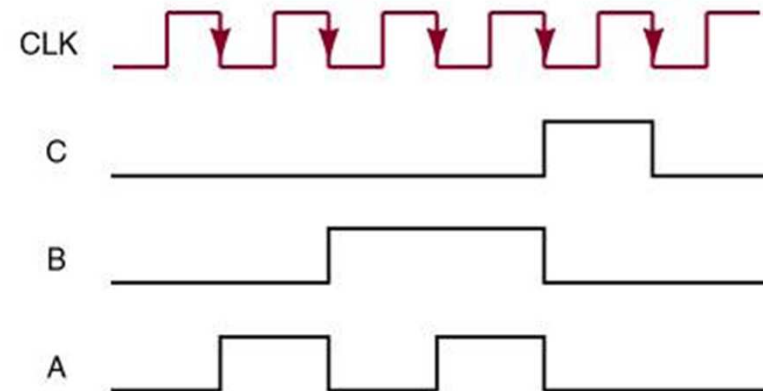
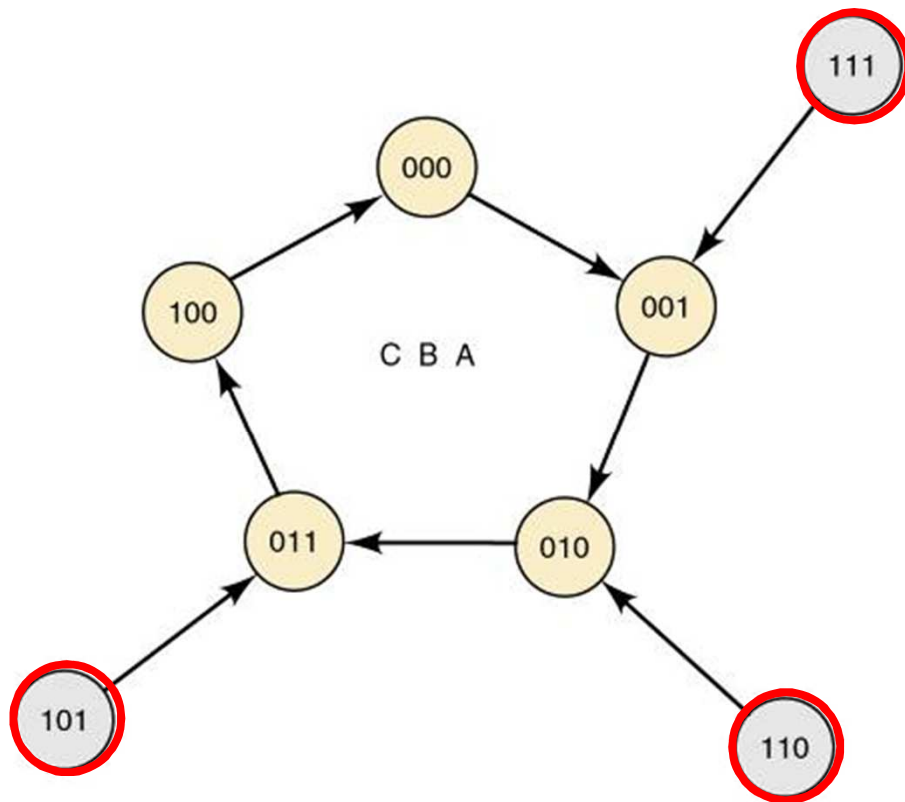
Analyzing Synchronous Counters

- Analyze counter designs of this type by predicting FF control inputs for each state of the counter.
 - A **PRESENT state/NEXT state table** is very useful for this purpose.

PRESENT State			Control Inputs						NEXT State		
<i>C</i>	<i>B</i>	<i>A</i>	<i>J_C</i>	<i>K_C</i>	<i>J_B</i>	<i>K_B</i>	<i>J_A</i>	<i>K_A</i>	<i>C</i>	<i>B</i>	<i>A</i>
0	0	0	0	0	0	0	1	1	0	0	1
0	0	1	0	0	1	1	1	1	0	1	0
0	1	0	0	0	0	0	1	1	0	1	1
0	1	1	1	0	1	1	1	1	1	0	0
1	0	0	0	1	0	0	0	0	0	0	0
1	0	1	0	1	1	1	0	0	0	1	1
1	1	0	0	1	0	0	0	0	0	1	0
1	1	1	1	1	1	1	0	0	0	0	1

Analyzing Synchronous Counters

Synchronous up counter state transition diagram and waveform.

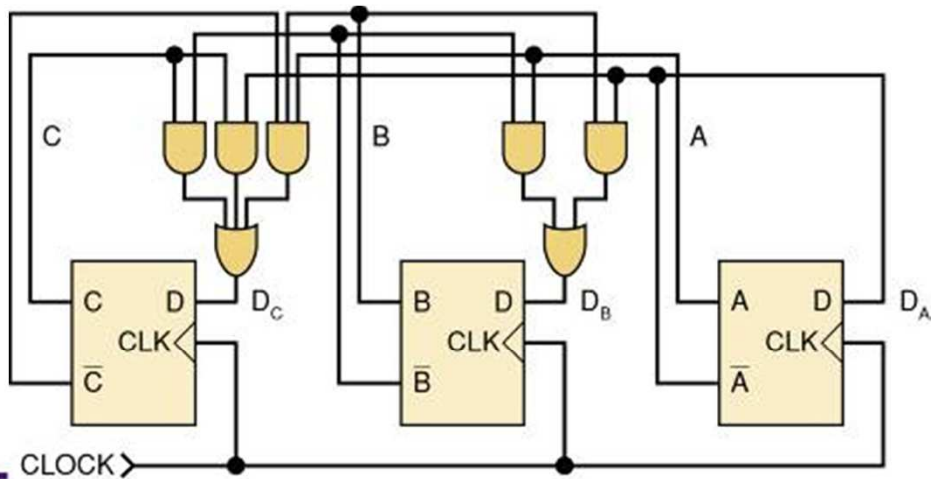


Highlighted information indicates this counter design is self-correcting.

Analyzing Synchronous Counters

Synchronous counter built using D-type FFs.

Control circuitry will typically be more complex than an equivalent JK-type counter,



PRESENT State			Control Inputs			NEXT State		
<i>C</i>	<i>B</i>	<i>A</i>	<i>D_C</i>	<i>D_B</i>	<i>D_A</i>	<i>C</i>	<i>B</i>	<i>A</i>
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	0
0	1	0	0	1	1	0	1	1
0	1	1	1	0	0	1	0	0
1	0	0	1	0	1	1	0	1
1	0	1	1	1	0	1	1	0
1	1	0	1	1	1	1	1	1
1	1	1	0	0	0	0	0	0

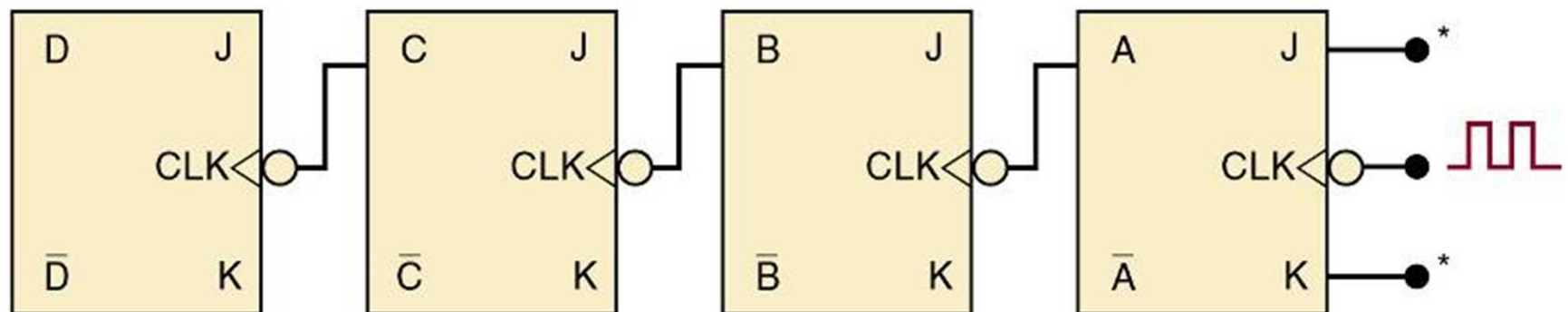
The number of synchronous inputs to control is reduced by half

Synchronous Counter Design

- Common design method using J-K flip-flops in a synchronous counter configuration.
 - Determine the desired number of bits (FFs) and the desired counting sequence.
 - Draw the state transition diagram showing *all* possible states—including those not part of the desired counting sequence.
 - Use the state transition diagram to set up a table that lists *all* PRESENT states and their NEXT states.
 - Add a column for each *J* and *K* input & indicate levels required to produce transition to the NEXT state.
 - Design the logic circuits to generate levels required at each input, and implement the final expressions.

IC Asynchronous Counter

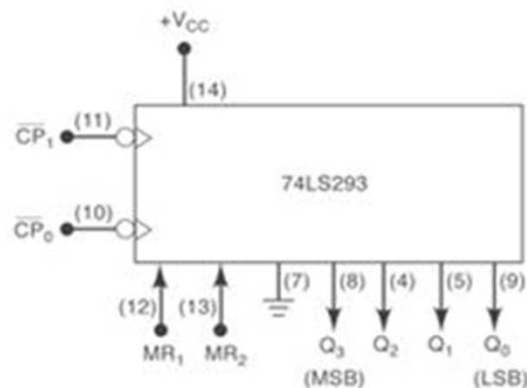
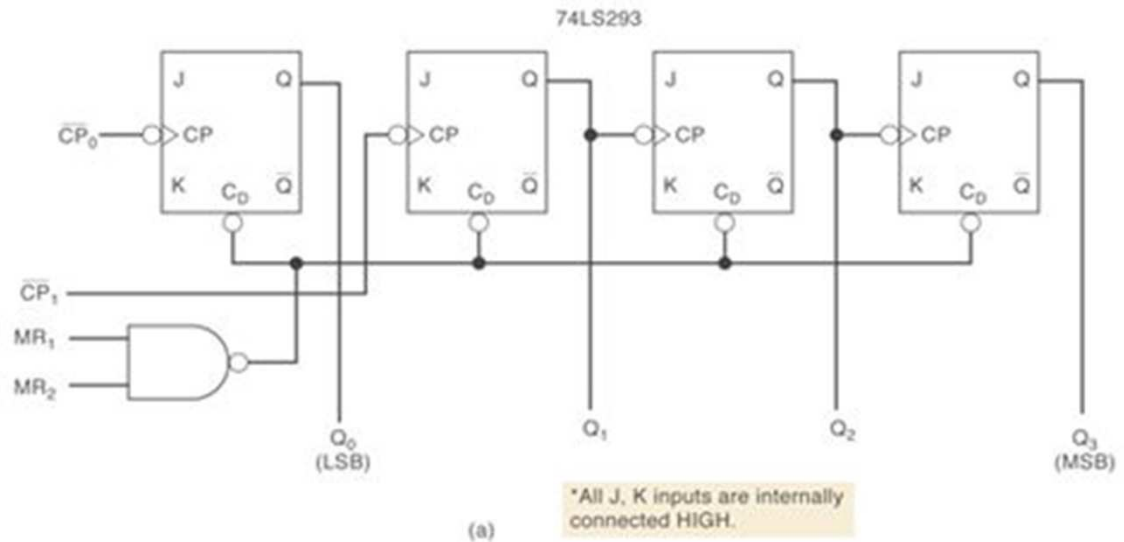
- Review of four bit counter operation:
 - Clock is applied only *CLK* input to FF *A*.
 - J & K are HIGH in all FFs.
 - Output of FF *A* is *CLK* of FF *B*, etc.
 - FF outputs *D*, *C*, *B*, and *A*, are a 4-bit binary number with *D* as the MSB.



*All J and K inputs
assumed to be 1.

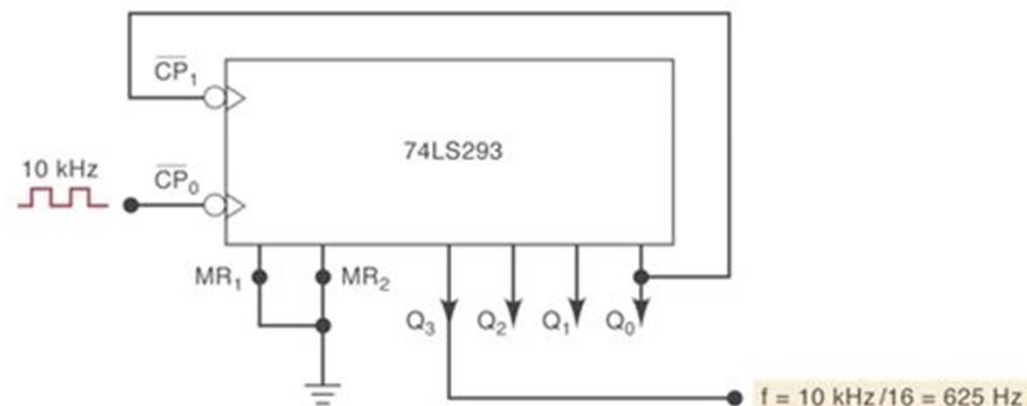
IC Asynchronous Counter

There are several TTL and CMOS asynchronous counter ICs. One of them is the TTL 74LS293



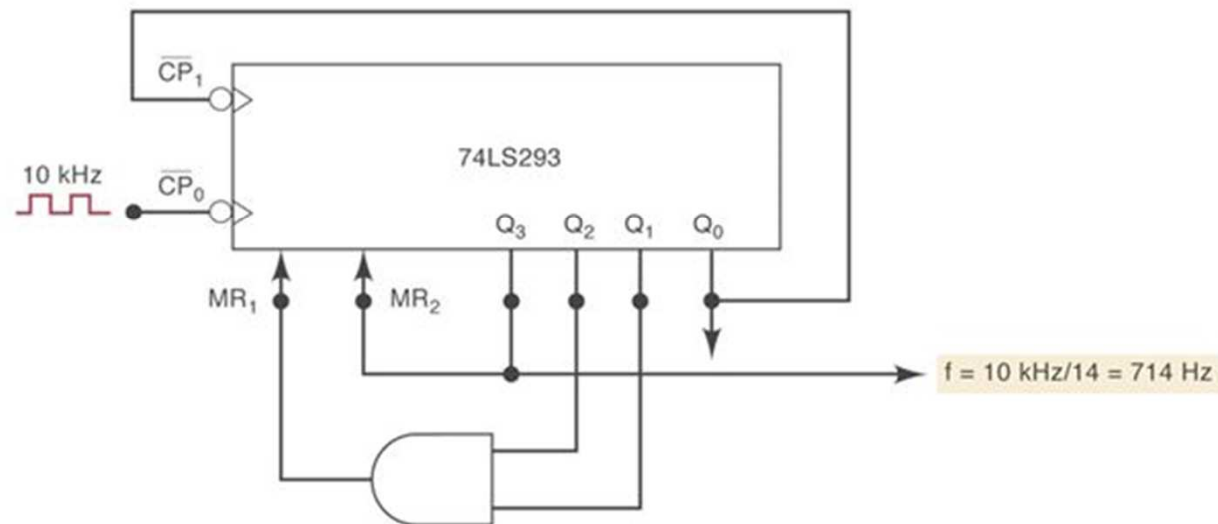
Example 1

- Show how the 74LS293 should be connected to operate as a MOD-16 counter with a 10-kHz clock input. Determine the frequency at Q_3 .
- MOD-16 needs 4 FFs. Therefore, The output of Q_0 must be connected to the next FF. Assume a clock frequency of 10 kHz, therefore Frequency at Q_3 would equal $10\text{kHz}/16 = 625\text{ Hz}$



Example 2

- Show how to wire the 74LS293 as a MOD-14 counter.
- When the counter reaches (14) 1110 it should reset. That is when $Q_1=Q_2=Q_3=1$. But the built-in NAND gate has only 2 inputs. Therefore, an extra AND gate must be added to have all the three inputs fed together.



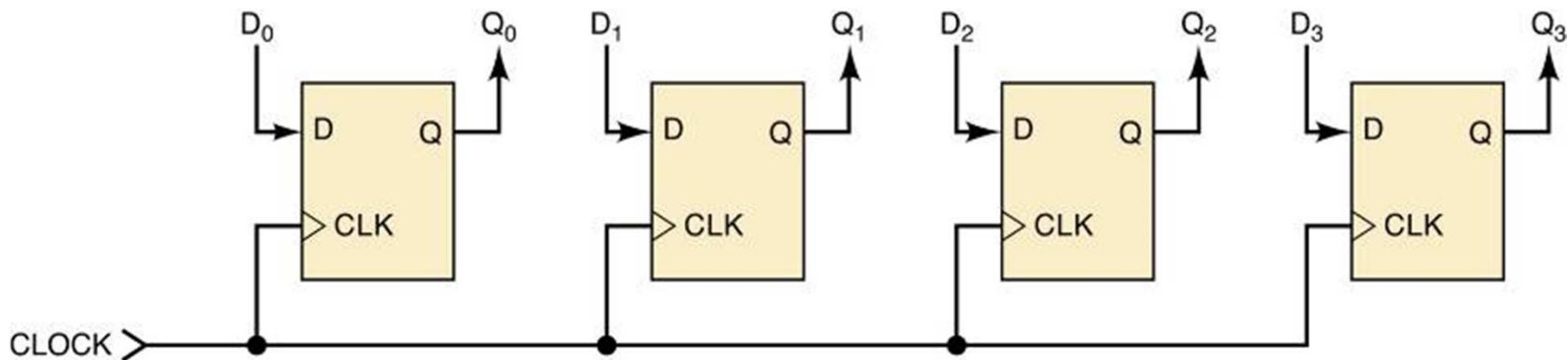
Register Data Transfer

- The various types of registers can be classified according to...
 - The manner in which data can be entered into the register for storage.
 - The manner in which data are outputted from the register.
- Serial data flow through a register is generally called *shifting*—either to the left or to the right.
 - Serial output data fed back to the input of the same register is called a *data rotate*.
- Parallel inputting of data is often described as a *register load*.

Register Data Transfer / Integrated-Circuit Registers

74ALS174/74HC174

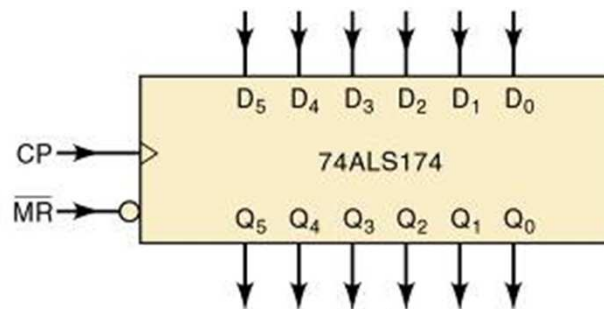
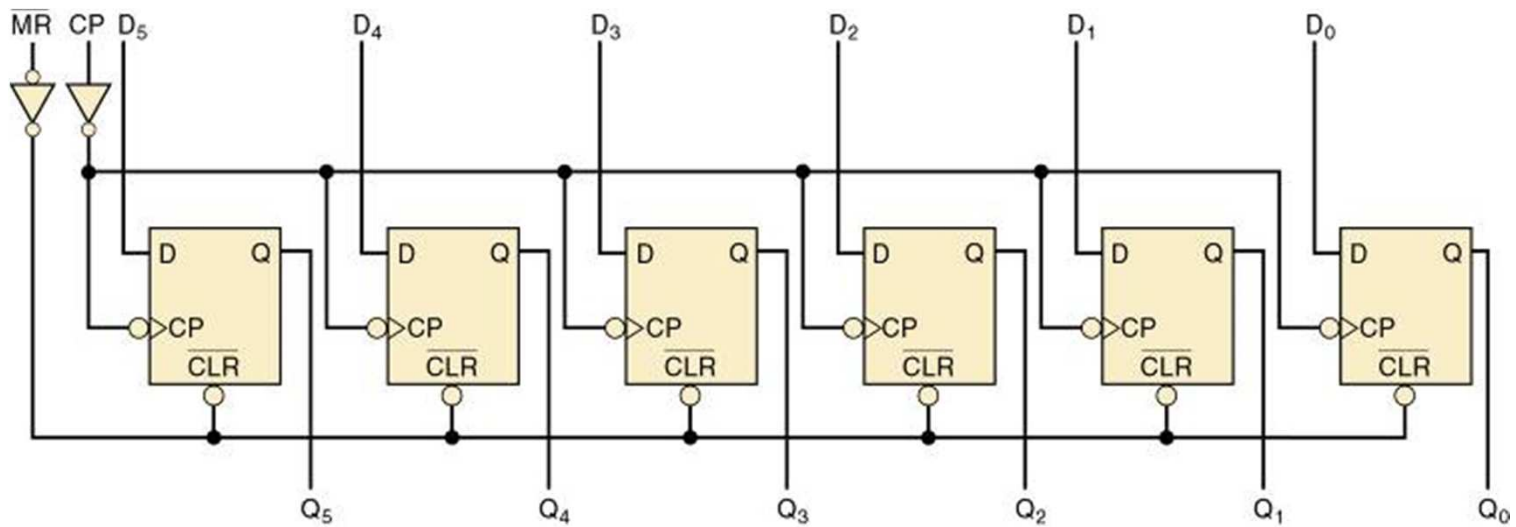
Parallel in/parallel out (PIPO)



A group of flip-flops that can store multiple bits simultaneously and in which all bits of the stored binary value are directly available.

Register Data Transfer / Integrated-Circuit Registers

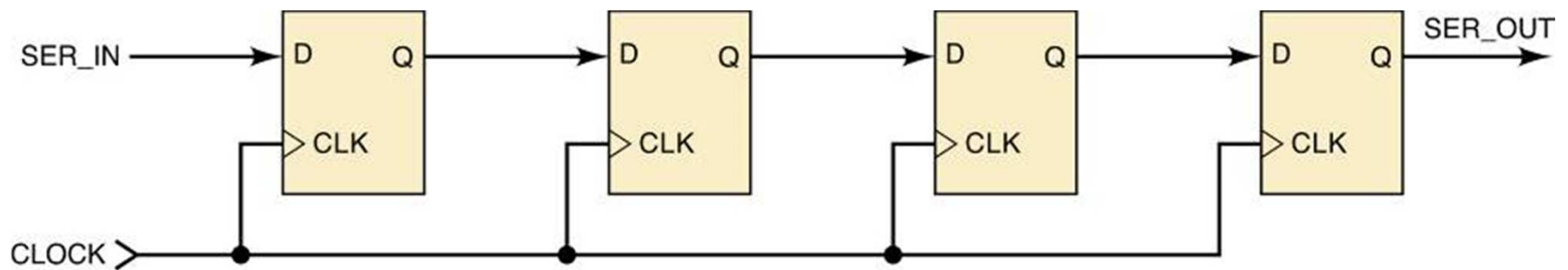
74ALS174/74HC174



Register Data Transfer / Integrated-Circuit Registers

74ALS166/74HC166

Serial in/serial out (SISO)

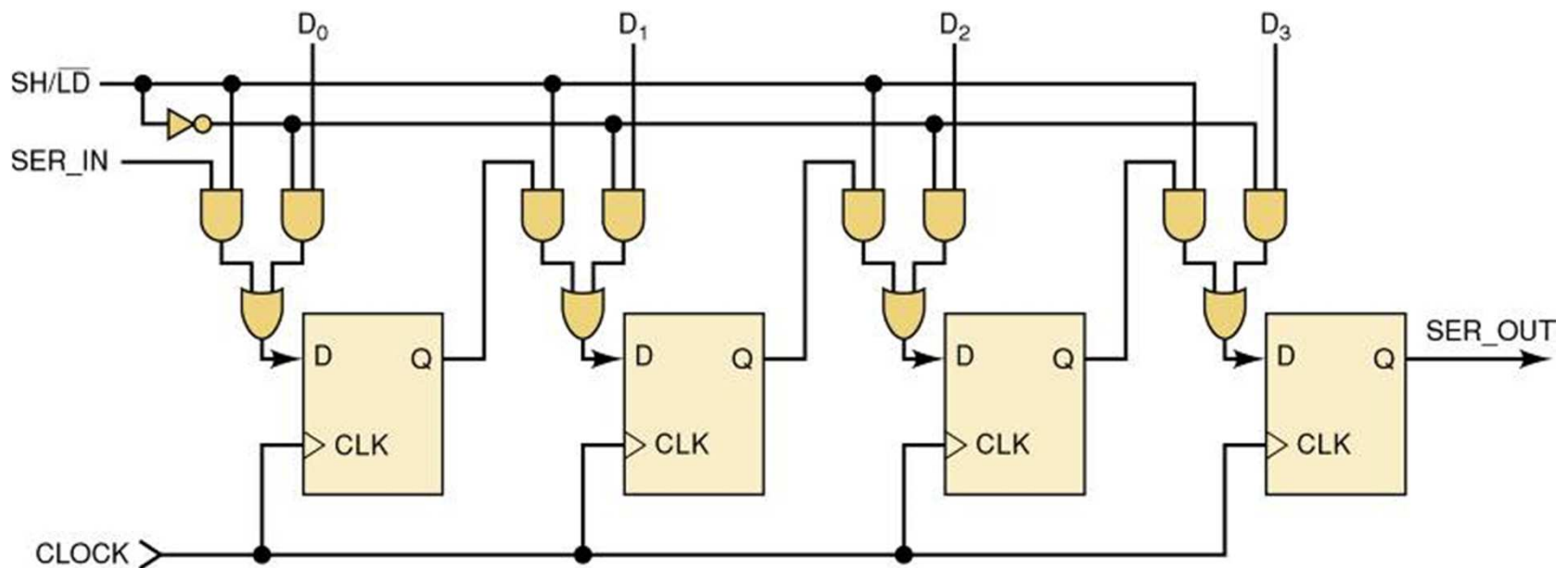


Data loaded one bit at a time moves one bit at a time, with each clock pulse through the flip-flops toward the other end of the register, and exit one bit at a time in the same order as originally loaded.

Register Data Transfer / Integrated-Circuit Registers

74ALS165/74HC165

8-bit parallel in/serial out (PISO)

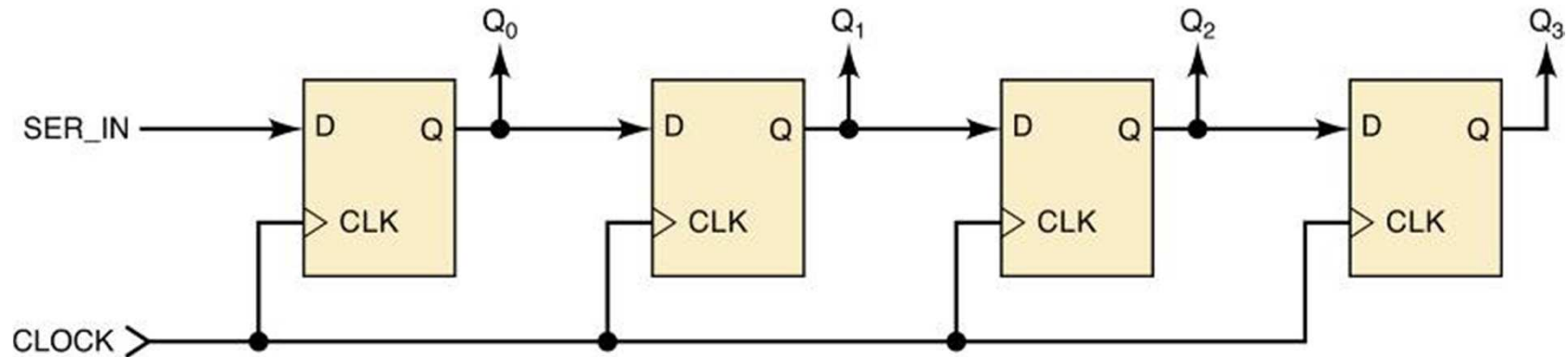


Serial shifting is always synchronous, as the clock is required to ensure the input data moves only one bit at a time with each appropriate clocking edge.

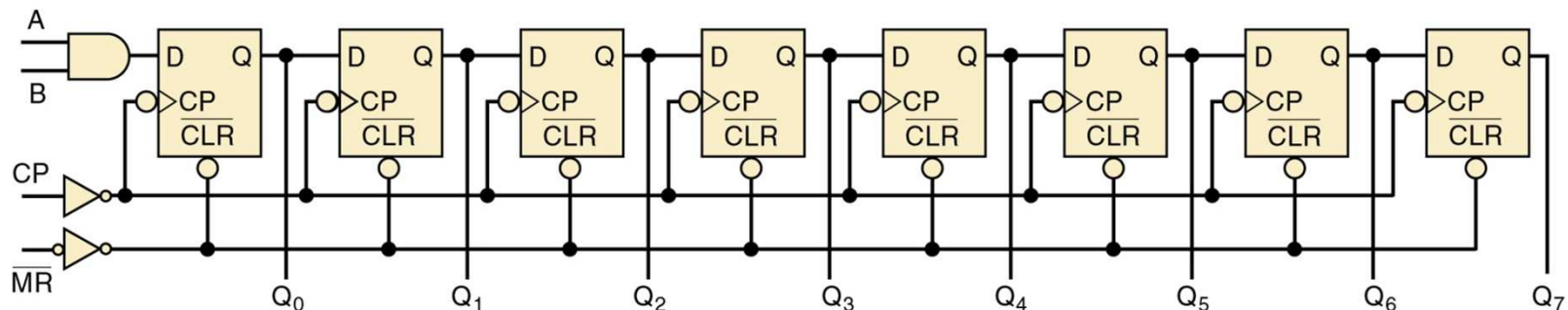
Register Data Transfer / Integrated-Circuit Registers

74ALS164/74HC164

8-bit serial in/parallel out (SIPO)



8-bit
shift register
74ALS164

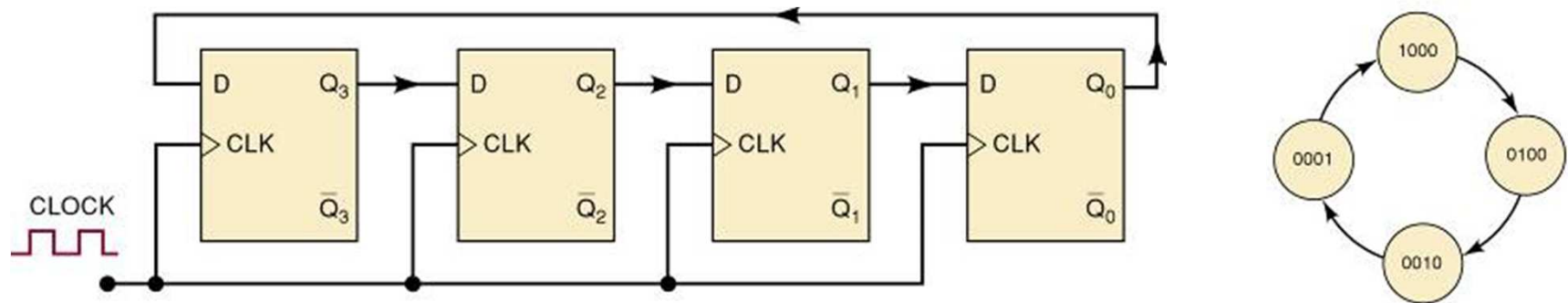


Shift Register Counters

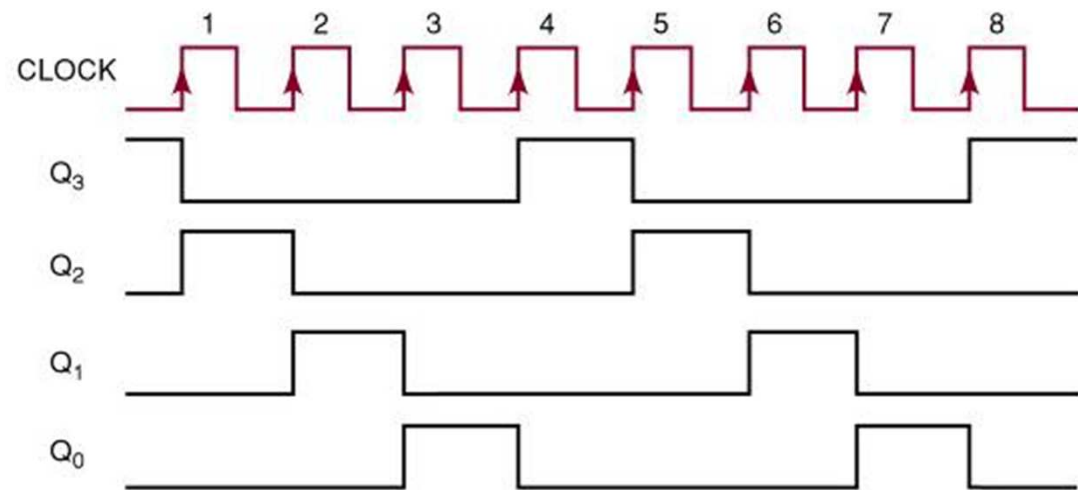
- Shift-register counters use *feedback*—the output of the last FF in the register is connected back to the first FF in some way.

Shift Register Counters

- A **ring counter** is a **circulating shift register** connected so the last FF shifts its value to the first.



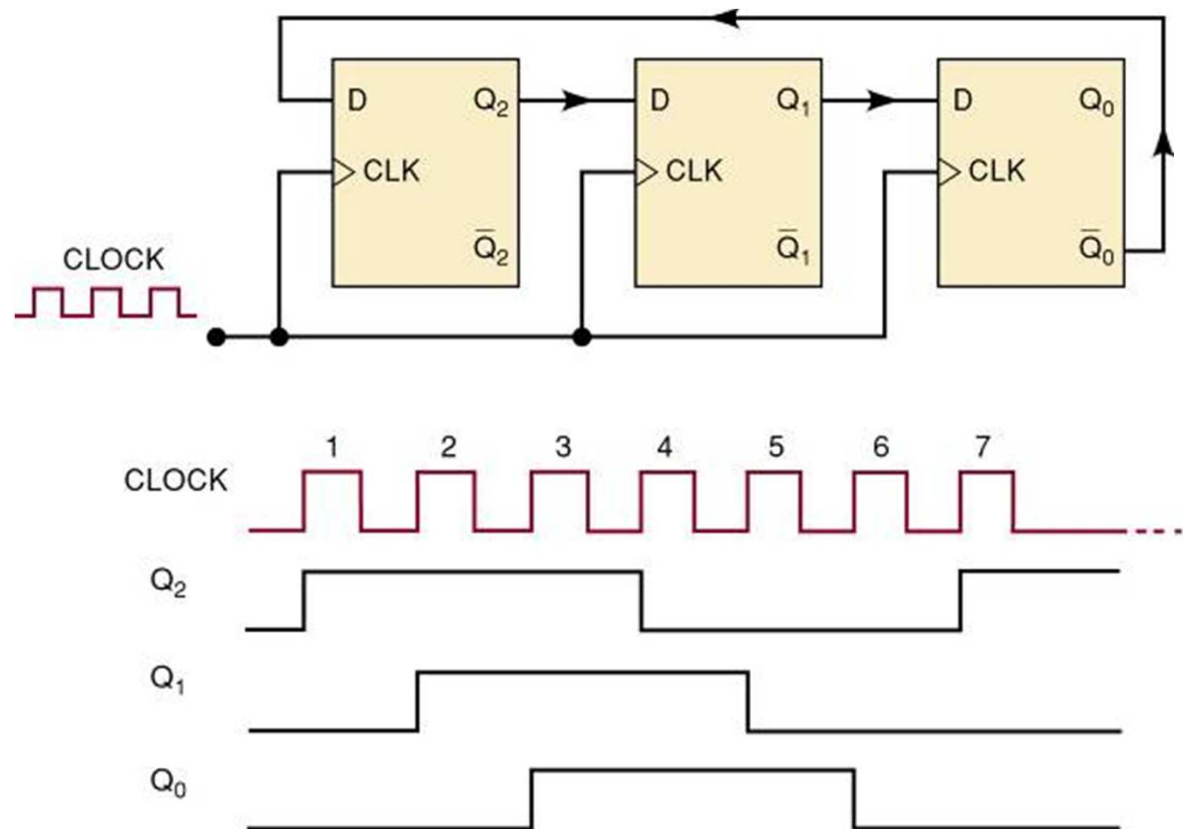
Q ₃	Q ₂	Q ₁	Q ₀	CLOCK pulse
1	0	0	0	0
0	1	0	0	1
0	0	1	0	2
0	0	0	1	3
1	0	0	0	4
0	1	0	0	5
0	0	1	0	6
0	0	0	1	7
.	.	.	.	.
.	.	.	.	.



Shift Register Counters

- In the **Johnson** or **twisted-ring counter** *inverted* output of the last FF is connected to the input of the first FF.

Q ₂	Q ₁	Q ₀	CLOCK pulse
0	0	0	0
1	0	0	1
1	1	0	2
1	1	1	3
0	1	1	4
0	0	1	5
0	0	0	6
1	0	0	7
1	1	0	8
·	·	·	·
·	·	·	·
·	·	·	·



Thank You



Next topic